

CASO PRÁCTICO 1 - UD4 - 2º DAM

José Cruces Aparicio

HISTÓRICO DE VISITAS

Contexto

El jefe del departamento de ventas de la empresa en la que trabajas pretende tener un histórico de las visitas realizadas a los clientes. Para ello, tu jefe te plantea añadir a la tabla usuarios una nueva columna que almacene un array de fechas de visitas a dicho cliente.

Cuestiones a resolver

Teniendo las siguientes especificaciones:

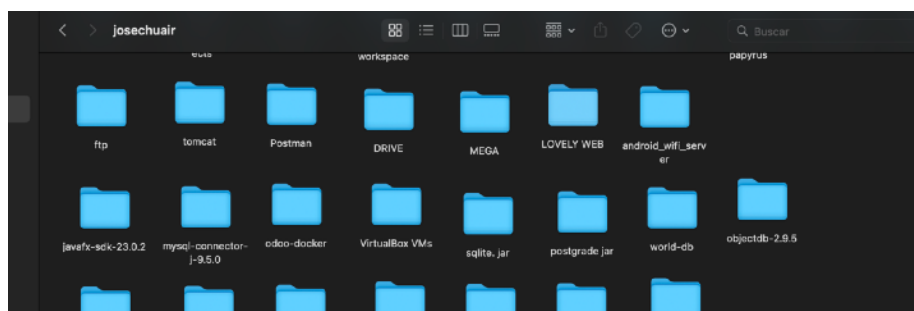
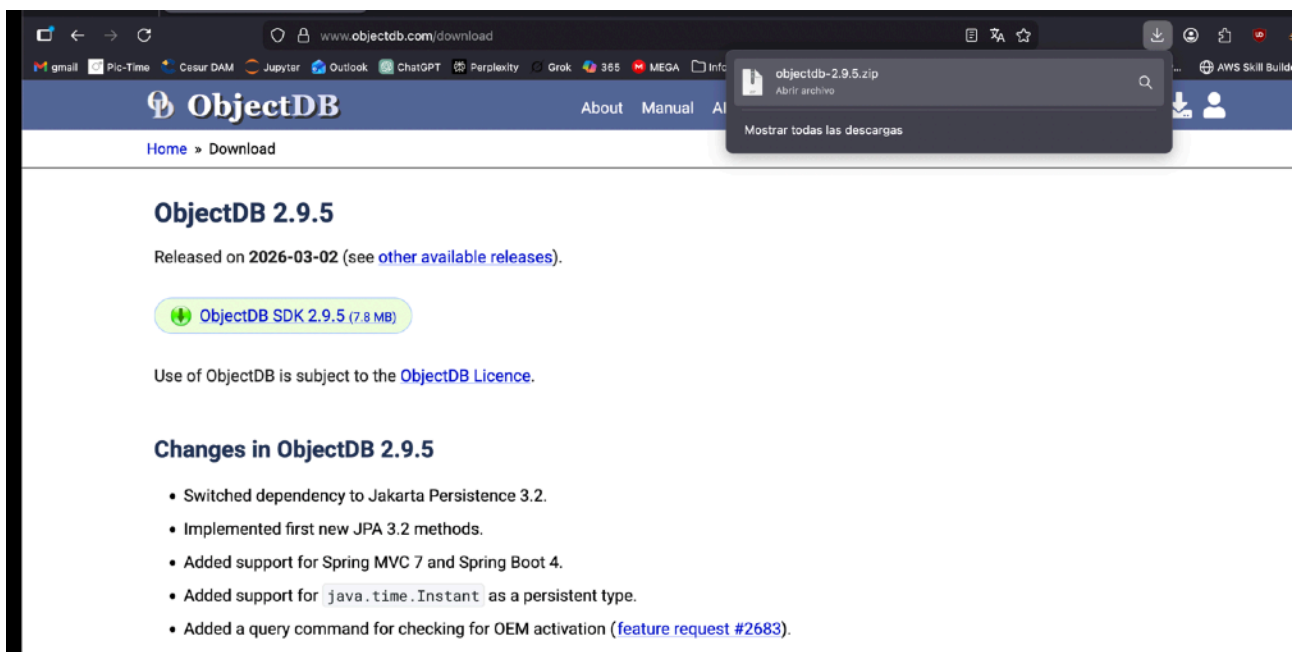
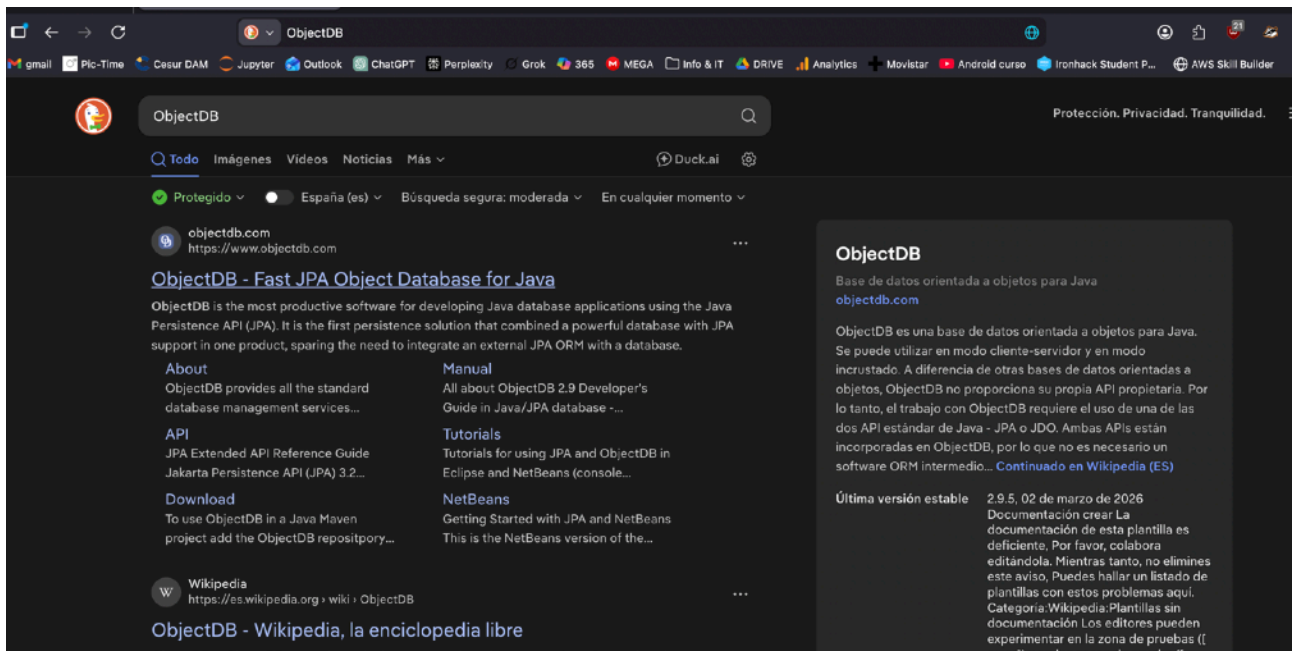
- o Base de datos ObjectDB.
- o Aplicación realizada en Java.
- o Clase cliente contiene:
 - Id.
 - Nombre.
 - Apellido 1.
 - Apellido 2.
 - Comercial principal.
 - Id empresa.

Realiza las siguientes tareas:

1. Crea la base de datos en ObjectDB.
2. Crea la clase de clientes, añadir el campo necesario para almacenar el array de visitas.
3. Crea el proyecto en Eclipse.
4. Crea las clases:
 - a. Una clase que permita las conexiones con la BBDD.
 - b. Una clase que modele la clase de la tabla.
 - c. Una clase que permita la interacción con el usuario.
 - d. Una clase que permita realizar un CRUD con la tabla.
5. Por último, crear la clase que permita probar toda la aplicación.

1. Creación de la base de datos en ObjectDB

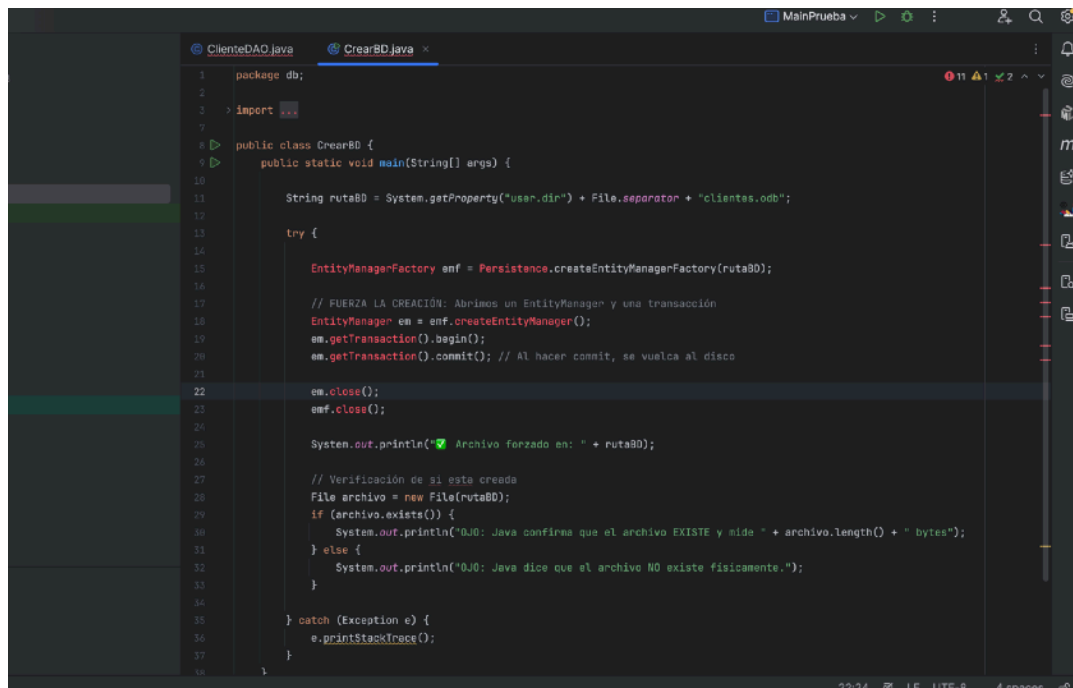
Primero se descarga ObjectDB desde su página oficial y se añade el archivo objectdb.jar al proyecto Java en la carpeta /lib .



Después se crea la clase CrearBD.java, cuya función es generar el archivo de base de datos (clientes.odb).

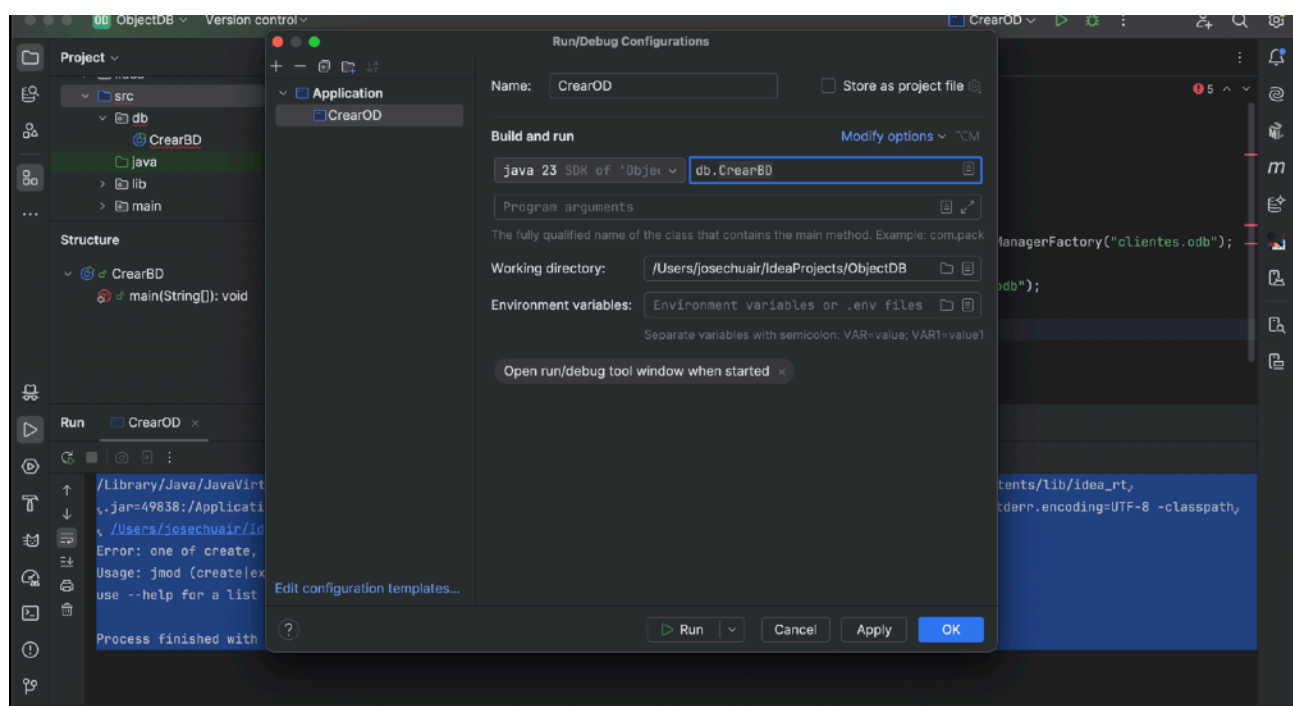
Proceso que realiza la clase:

1. Define la ruta donde se crea la base de datos.
2. Crea un EntityManagerFactory .
3. Abre una transacción para forzar la creación del archivo.
4. Comprueba si el archivo existe físicamente.

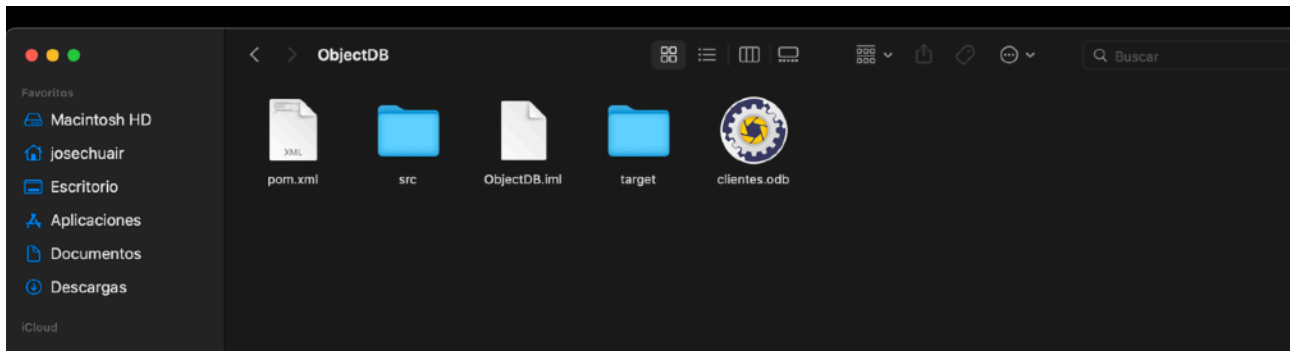
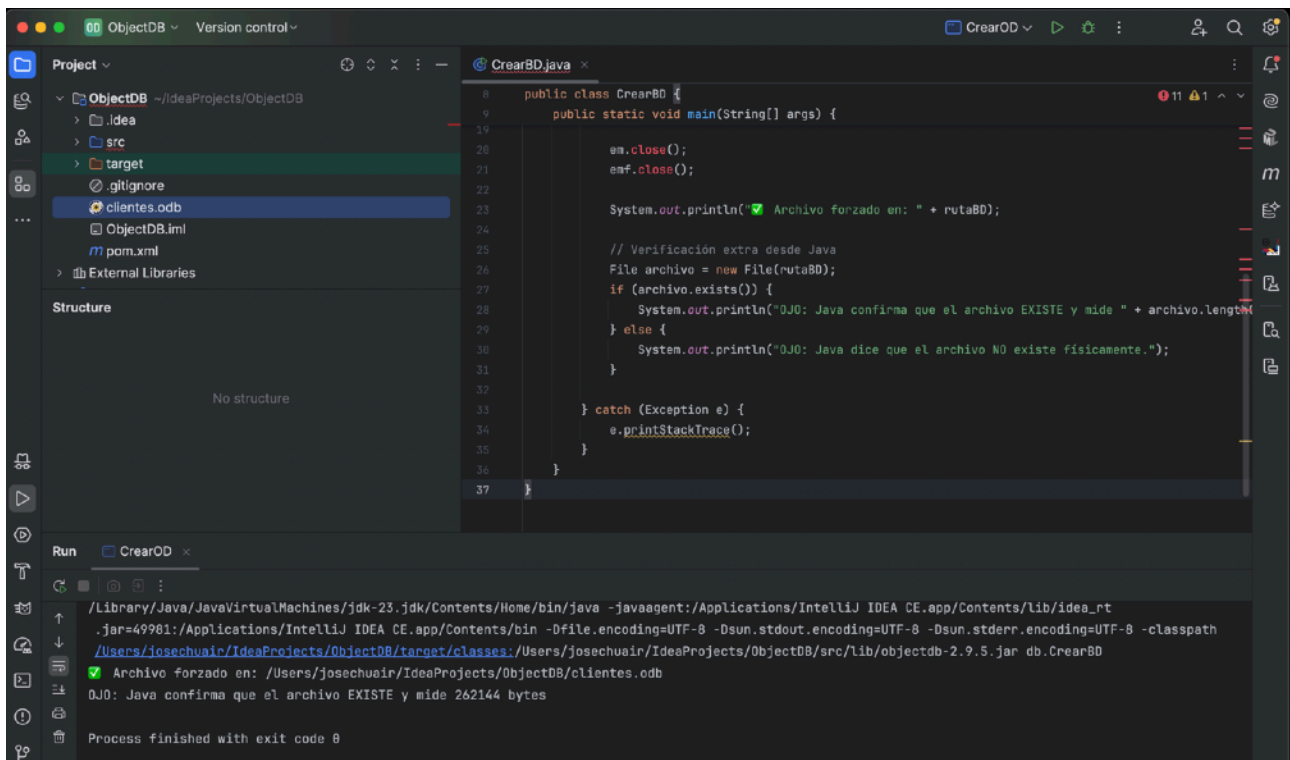


```
1 package db;
2
3 import javax.persistence.*;
4
5 public class CrearBD {
6     public static void main(String[] args) {
7
8         String rutaBD = System.getProperty("user.dir") + File.separator + "clientes.odb";
9
10        try {
11
12            EntityManagerFactory emf = Persistence.createEntityManagerFactory(rutaBD);
13
14            // FUERZA LA CREACION: Abrimos un EntityManager y una transacción
15            EntityManager em = emf.createEntityManager();
16            em.getTransaction().begin();
17            em.getTransaction().commit(); // Al hacer commit, se vuelca al disco
18
19            em.close();
20            emf.close();
21
22            System.out.println("✅ Archivo forzado en: " + rutaBD);
23
24            // Verificación de si esta creada
25            File archivo = new File(rutaBD);
26            if (archivo.exists()) {
27                System.out.println("OBJ: Java confirma que el archivo EXISTE y mide " + archivo.length() + " bytes");
28            } else {
29                System.out.println("OBJ: Java dice que el archivo NO existe físicamente.");
30            }
31
32        } catch (Exception e) {
33            e.printStackTrace();
34        }
35    }
36 }
```

Editamos la configuración para ejecutar la clase y que cree la bd.



Mostramos la correcta creación de la BD .



2. Creación de la clase Cliente (modelo cliente.java)

La clase Cliente.java representa la tabla de clientes dentro de la base de datos. Se utiliza la anotación:

Java

@Entity

para indicar que la clase será persistente.

Atributos de la clase

id

nombre

apellido1

apellido2

comercialPrincipal

idEmpresa

visitas (array de fechas)

El requisito del ejercicio es guardar un histórico de visitas, por lo que se añade: Java
private Date[] visitas;

Esto permite almacenar varias fechas de visitas realizadas a un cliente. También incluye:

- Constructor vacío
- Constructor completo
- Getters y setters
- Método toString() para mostrar la información.

Ciente.java

```
package modelo;
```

```
import javax.persistence.Entity;  
import javax.persistence.Id;  
import java.io.Serializable;  
import java.util.Date;  
import java.util.Arrays;
```

```
@Entity
```

```
public class Ciente implements Serializable {
```

```
    @Id
```

```
    private int id;
```

```
    private String nombre;
```

```
    private String apellido1;
```

```
    private String apellido2;
```

```
    private String comercialPrincipal;
```

```
    private int idEmpresa;
```

```
    // Histórico de visitas
```

```
    private Date[] visitas;
```

```
    public Ciente() {}
```

```
    public Ciente(int id, String nombre, String apellido1, String apellido2, String comercialPrincipal,  
int idEmpresa, Date[] visitas) {
```

```
        this.id = id;
```

```
        this.nombre = nombre;
```

```
        this.apellido1 = apellido1;
```

```
        this.apellido2 = apellido2;
```

```
        this.comercialPrincipal = comercialPrincipal;
```

```
        this.idEmpresa = idEmpresa;
```

```
        this.visitas = visitas;
```

```
    }
```

```
    // Getters y Setters
```

```
    public int getId()
```

```
    {
```

```
        return id;
```

```
    }
```

```
    public void setId(int id)
```

```
    {
```

```
        this.id = id;
```

```
    }
```

```
    public String getNombre()
```

```
    {
```

```
        return nombre;
```

```
    }
```

```

public void setNombre(String nombre)
{
    this.nombre = nombre;
}
public Date[] getVisitas()
{
    return visitas;
}
public void setVisitas(Date[] visitas)
{
    this.visitas = visitas;
}

@Override
public String toString() {
    return String.format("ID: %d | %s %s | Comercial: %s | Visitas: %s",
        id, nombre, apellido1, comercialPrincipal, Arrays.toString(visitas));
}
}

```

3. Creación del proyecto en Eclipse

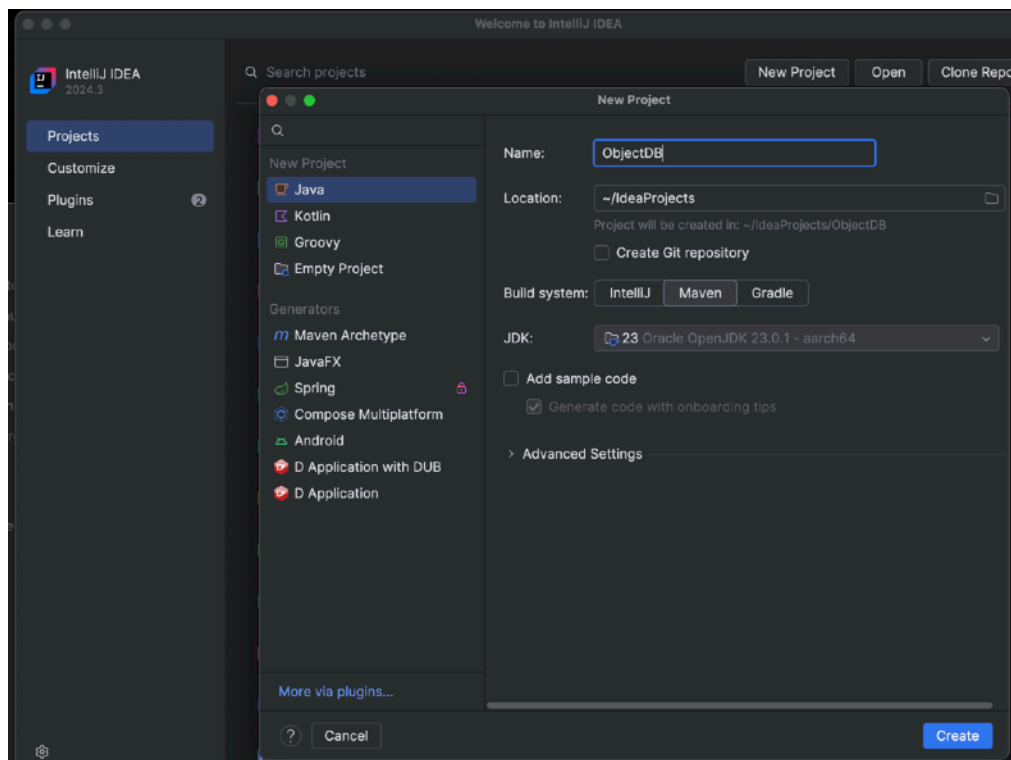
Para el desarrollo se crea un proyecto Java con Maven. Dentro del proyecto se añaden:

Carpetas o paquetes:
 db
 modelo
 main

Además, se copia en la carpeta /lib los siguientes archivos:

- objectdb.jar
- javax.persistence
- java.transaction

Estos permiten trabajar con JPA y ObjectDB.



4. Clases necesarias de la aplicación

El programa se divide en varias clases para organizar mejor el código.

4.1 Clase de conexión a la base de datos

Clase: ConexionBD.java

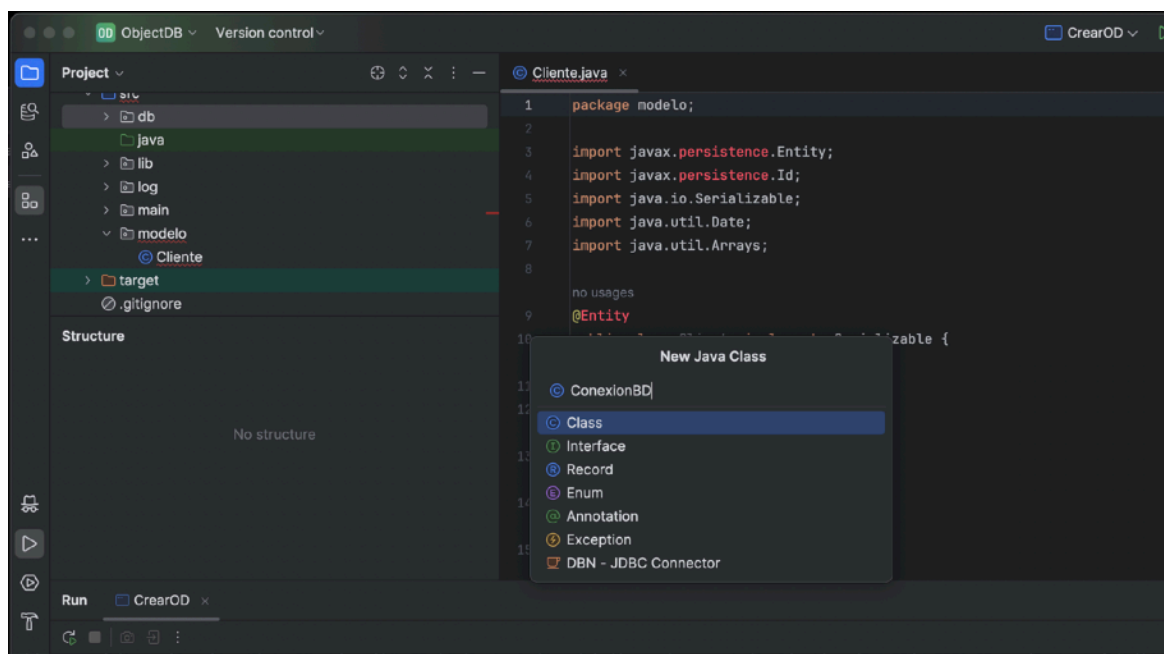
Esta clase se encarga de gestionar la conexión con la base de datos.

Funciones principales:

Crear el EntityManagerFactory

Proporcionar acceso a la base de datos

Cerrar la conexión cuando termina el programa



ConexionBD.java

```
package db;
```

```
import javax.persistence.EntityManagerFactory;  
import javax.persistence.Persistence;  
import java.io.File;
```

```
public class ConexionBD {  
    private static EntityManagerFactory emf;  
    private static final String URL = System.getProperty("user.dir") + File.separator + "clientes.odb";  
  
    public static EntityManagerFactory getFactory() {  
        if (emf == null || !emf.isOpen()) {
```

```

        emf = Persistence.createEntityManagerFactory(URL);
    }
    return emf;
}

public static void cerrar() {
    if (emf != null && emf.isOpen()) {
        emf.close();
    }
}
}

```

4.2 Clase DAO (CRUD ClienteDAO.java)

Clase: ClienteDAO.java

Esta clase contiene las operaciones para trabajar con la base de datos. CRUD significa:

- Create → insertar cliente
- Read → leer clientes
- Delete → eliminar cliente

Insertar cliente

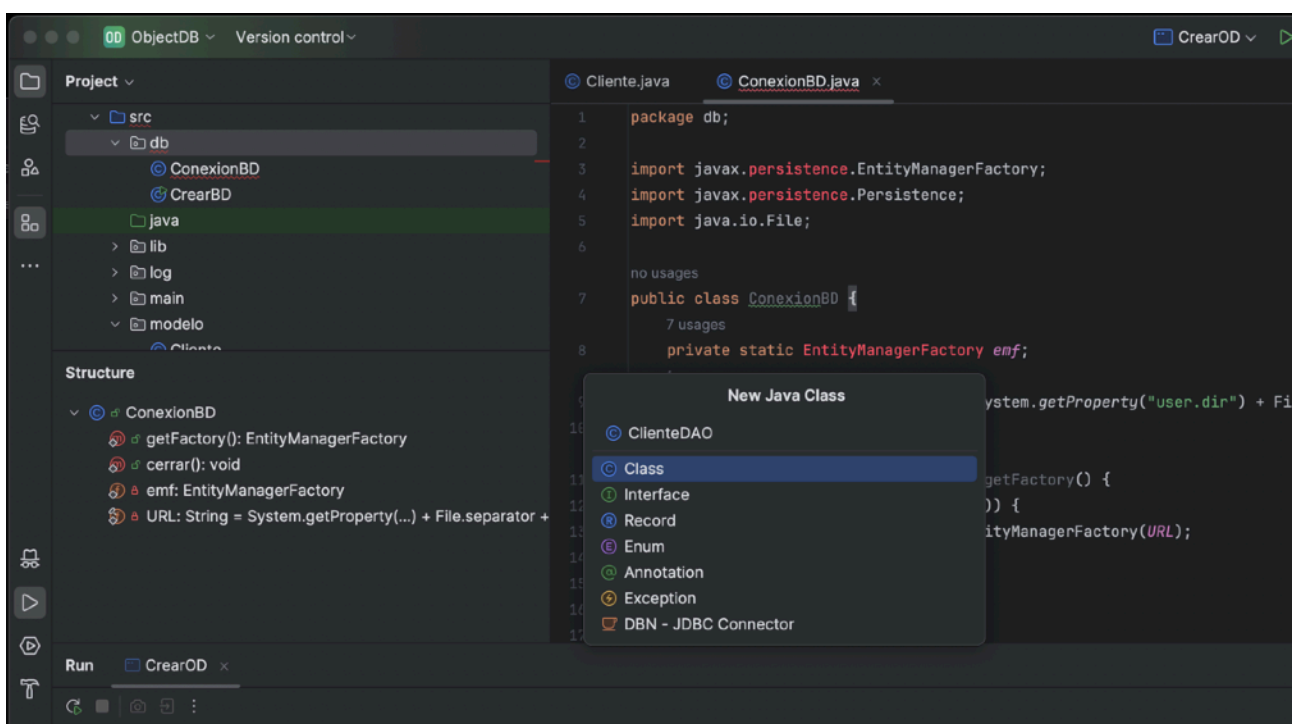
em.persist(c);

Guarda el objeto en la base de datos.

Leer todos los clientes

SELECT c FROM Cliente c

Consulta todos los registros.



Eliminar cliente

```
Cliente c = em.find(Cliente.class, id);  
em.remove(c);
```

Busca el cliente por id y lo elimina.

ClienteDAO.java

```
package db;
```

```
import modelo.Cliente;  
import javax.persistence.EntityManager;  
import javax.persistence.TypedQuery;  
import java.util.List;
```

```
public class ClienteDAO {
```

```
    public void insertar(Cliente c) {  
        EntityManager em = ConexionBD.getFactory().createEntityManager();  
        em.getTransaction().begin();  
        em.persist(c);  
        em.getTransaction().commit();  
        em.close();  
    }
```

```
    public List<Cliente> leerTodos() {  
        EntityManager em = ConexionBD.getFactory().createEntityManager();  
        TypedQuery<Cliente> query = em.createQuery("SELECT c FROM Cliente c", Cliente.class);  
        List<Cliente> lista = query.getResultList();  
        em.close();  
        return lista;  
    }
```

```
    public void eliminar(int id) {  
        EntityManager em = ConexionBD.getFactory().createEntityManager();  
        em.getTransaction().begin();  
        Cliente c = em.find(Cliente.class, id);  
        if (c != null) em.remove(c);  
        em.getTransaction().commit();  
        em.close();  
    }  
}
```

4.3 Interfaz de usuario (InterfazUsuario.java)

Clase: InterfazUsuario.java

Permite interactuar con el usuario mediante consola.

Se utiliza Scanner para pedir los datos del cliente:

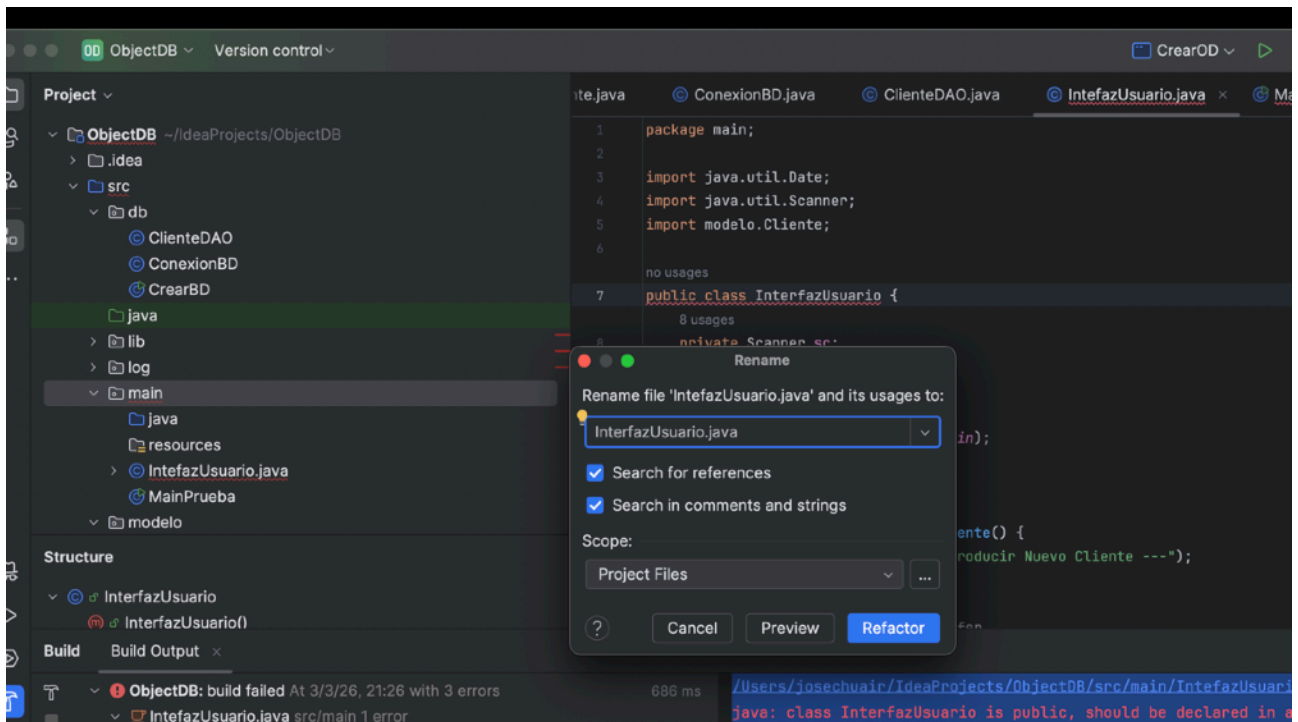
- id
- nombre
- apellidos
- comercial
- id empresa

Después crea un objeto Cliente.

Además, automáticamente se añade una fecha de visita:

```
Date[] visitas = { new Date() };
```

Esto registra la fecha actual como visita del cliente.



InterfazUsuario.java

```
package main;
```

```
import java.util.Date;  
import java.util.Scanner;  
import modelo.Cliente;
```

```
public class InterfazUsuario {  
    private Scanner sc;
```

```
    public InterfazUsuario() {  
        this.sc = new Scanner(System.in);  
    }
```

```
    public Cliente pedirDatosNuevoCliente() {  
        System.out.println("\n--- Introducir Nuevo Cliente ---");  
        System.out.print("ID: ");  
        int id = sc.nextInt();  
        sc.nextLine();  
        System.out.print("Nombre: ");  
        String nombre = sc.nextLine();  
        System.out.print("Apellido 1: ");  
        String ap1 = sc.nextLine();  
        System.out.print("Apellido 2: ");  
        String ap2 = sc.nextLine();  
        System.out.print("Comercial Principal: ");
```

```

String comercial = sc.nextLine();
System.out.print("ID Empresa: ");
int idEmpresa = sc.nextInt();

Date[] visitas = { new Date() };

return new Cliente(id, nombre, ap1, ap2, comercial, idEmpresa, visitas);
}

public void mostrarMensaje(String msj) {
    System.out.println(msj);
}
}

```

4.4 Clase de prueba (MainPrueba.java)

Clase: MainPrueba.java

Es la clase principal que ejecuta el programa.

Incluye un menú interactivo:

1. Añadir cliente
2. Ver clientes
3. Eliminar cliente
4. Salir

Dependiendo de la opción elegida:

- se inserta un cliente
- se muestran todos los clientes
- se elimina un cliente

Ejemplo de lectura:

```

List<Cliente> lista = dao.leerTodos();
for (Cliente c : lista)
    System.out.println(c);

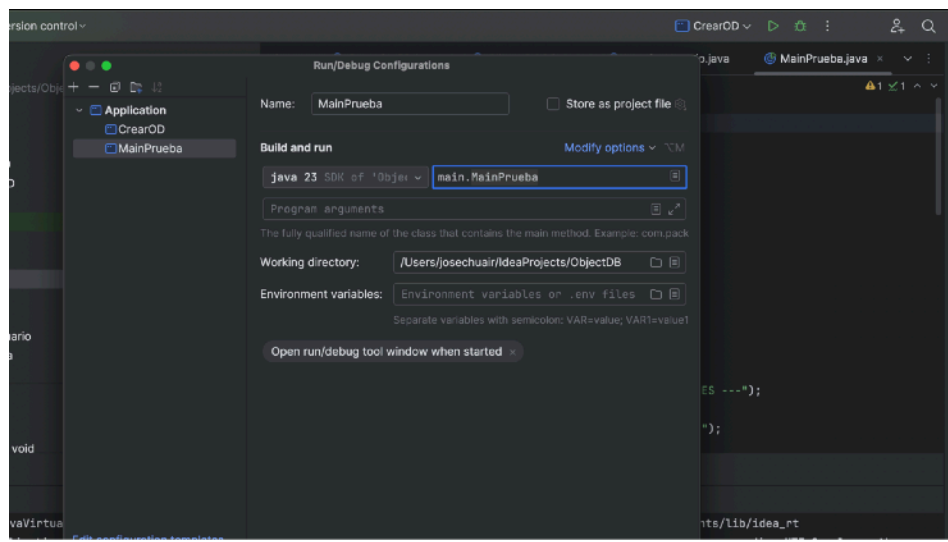
```

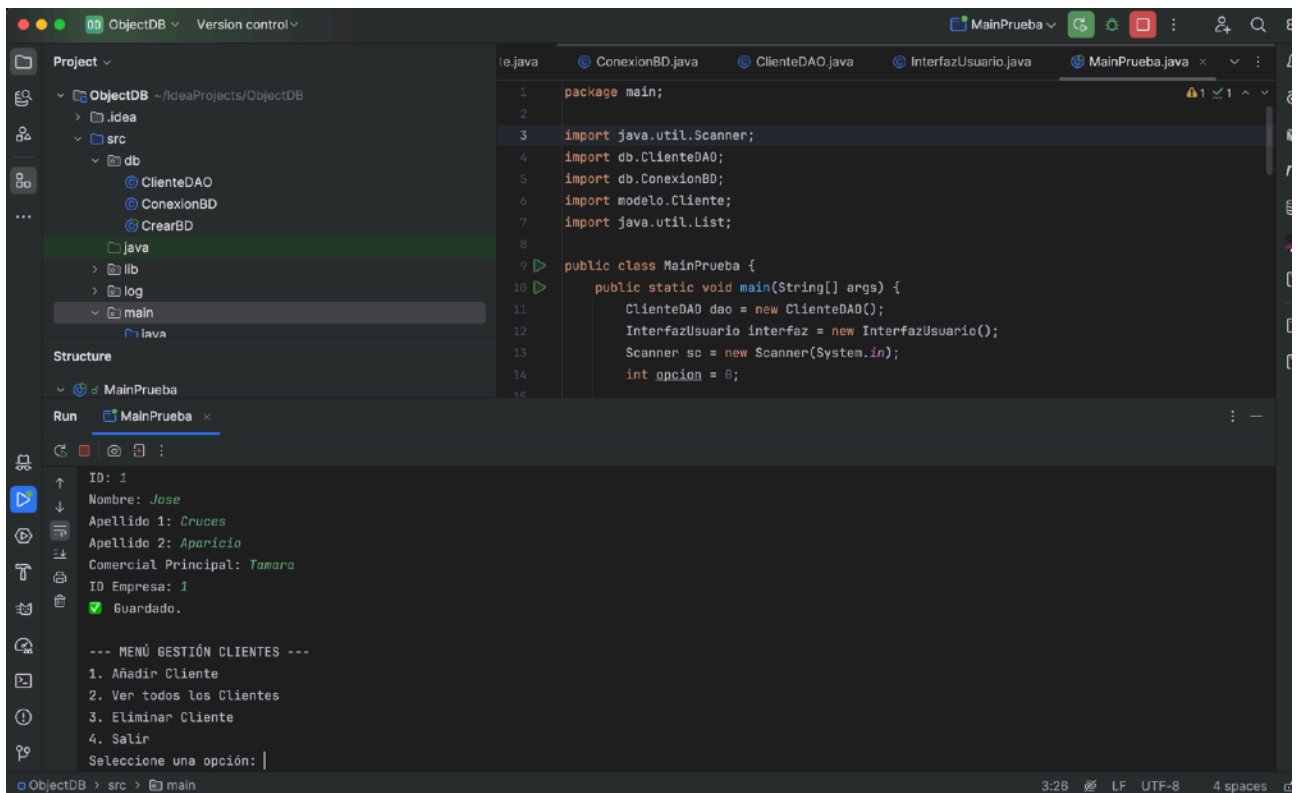
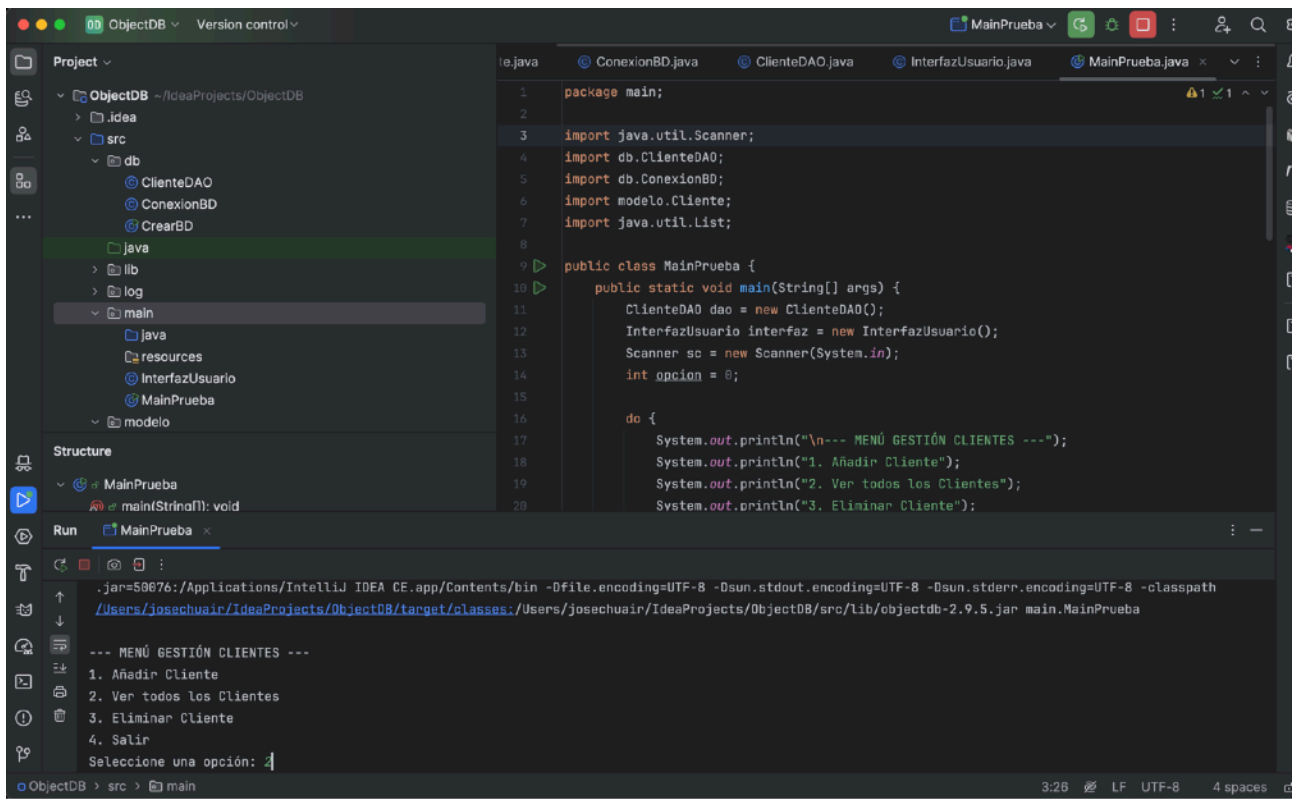
5. Prueba de la aplicación

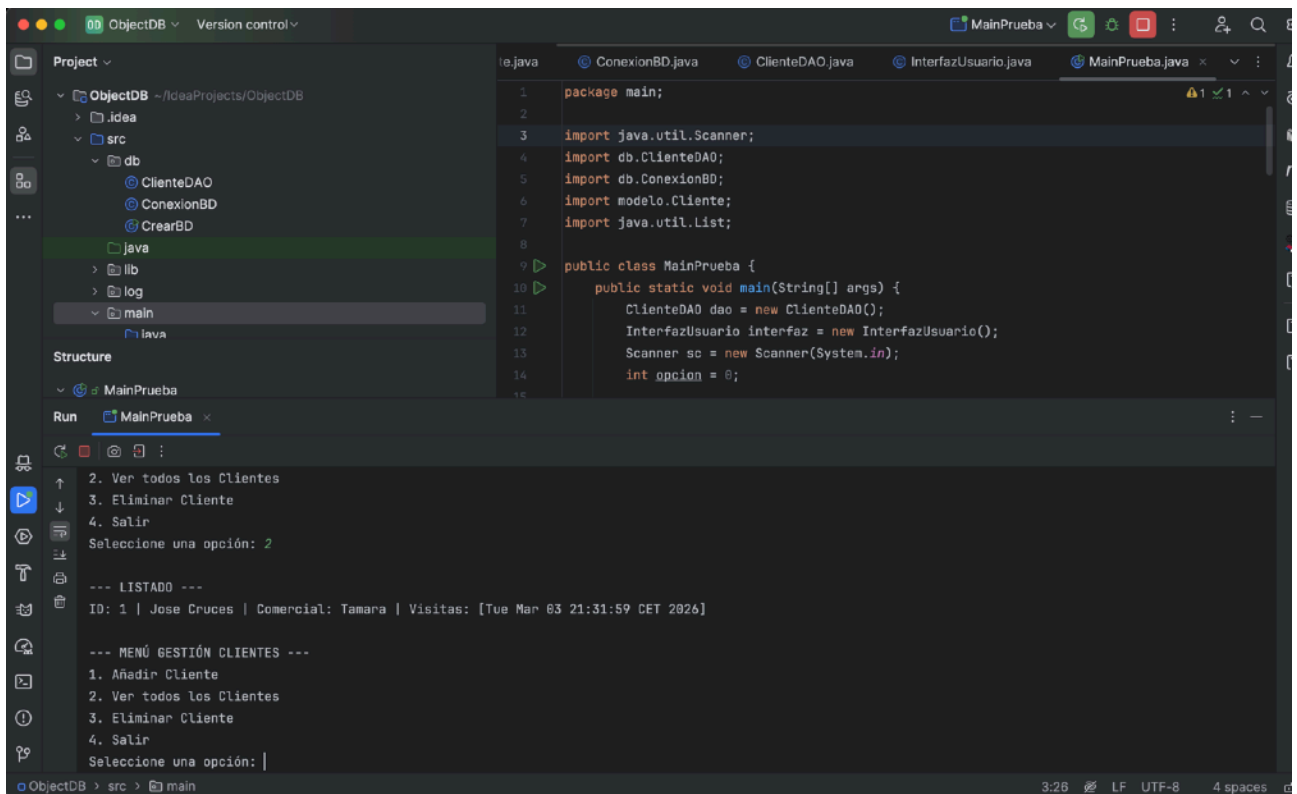
Al ejecutar el programa:

1. Aparece el menú.
2. Se pueden introducir clientes.
3. Los datos se guardan en la base de datos ObjectDB.
4. Se pueden visualizar todos los registros.
5. También se pueden eliminar clientes por ID.

Esto demuestra que el CRUD funciona correctamente.







MainPrueba.java

```
package main;
```

```
import java.util.Scanner;  
import db.ClienteDAO;  
import db.ConexionBD;  
import modelo.Cliente;  
import java.util.List;
```

```
public class MainPrueba {  
    public static void main(String[] args) {  
        ClienteDAO dao = new ClienteDAO();  
        InterfazUsuario interfaz = new InterfazUsuario();  
        Scanner sc = new Scanner(System.in);  
        int opcion = 0;  
  
        do {  
            System.out.println("\n--- MENÚ GESTIÓN CLIENTES ---");  
            System.out.println("1. Añadir Cliente");  
            System.out.println("2. Ver todos los Clientes");  
            System.out.println("3. Eliminar Cliente");  
            System.out.println("4. Salir");  
            System.out.print("Seleccione una opción: ");  
            opcion = sc.nextInt();  
  
            switch (opcion) {  
                case 1:  
                    Cliente nuevo = interfaz.pedirDatosNuevoCliente();  
                    dao.insertar(nuevo);  

```

```

        System.out.println("Nuevo cliente guardado.");
        break;
    case 2:
        System.out.println("\n--- LISTADO ---");
        List<Cliente> lista = dao.leerTodos();
        if (lista.isEmpty()) System.out.println("La base de datos está vacía.");
        for (Cliente c : lista) System.out.println(c);
        break;
    case 3:
        System.out.print("Introduce el ID a borrar: ");
        int id = sc.nextInt();
        dao.eliminar(id);
        System.out.println("🗑 Cliente eliminado.");
        break;
    case 4:
        System.out.println("Saliendo...");
        break;
    default:
        System.out.println("Opción no válida.");
    }
} while (opcion != 4);

ConexionBD.cerrar();
}
}

```

6. Conclusión

En este caso práctico se ha desarrollado una aplicación en Java con ObjectDB que permite gestionar clientes y almacenar el histórico de visitas mediante un array de fechas.

El sistema implementa:

- conexión a base de datos
- persistencia de objetos
- operaciones CRUD
- interacción con el usuario
- pruebas del sistema

Esto demuestra cómo trabajar con bases de datos orientadas a objetos usando JPA y ObjectDB.

Bibliografía

Ud 4 Acceso de datos

<https://www.objectdb.com/>